

임베디드 SW

2차 프로젝트

혁신공유학부, 차세대반도체전공

2025-51437 김동준

소속: 송실대학교 전자정보공학부(IT융합전공) 3학년

1. 시스템 개요

이번 프로젝트는 조명 제어 시스템을 구현하였다. ATtiny4313을 기반으로 로터리 엔코더와 PS/2 키보드를 통해 입력을 받고, OLED와 부저, PWM LED를 통해 시각/청각적 피드백을 제공한다. 또한, UART를 통한 관리자 콘솔을 제작하여 현재 상황 확인이 가능하다.

1.1. 핵심 기능

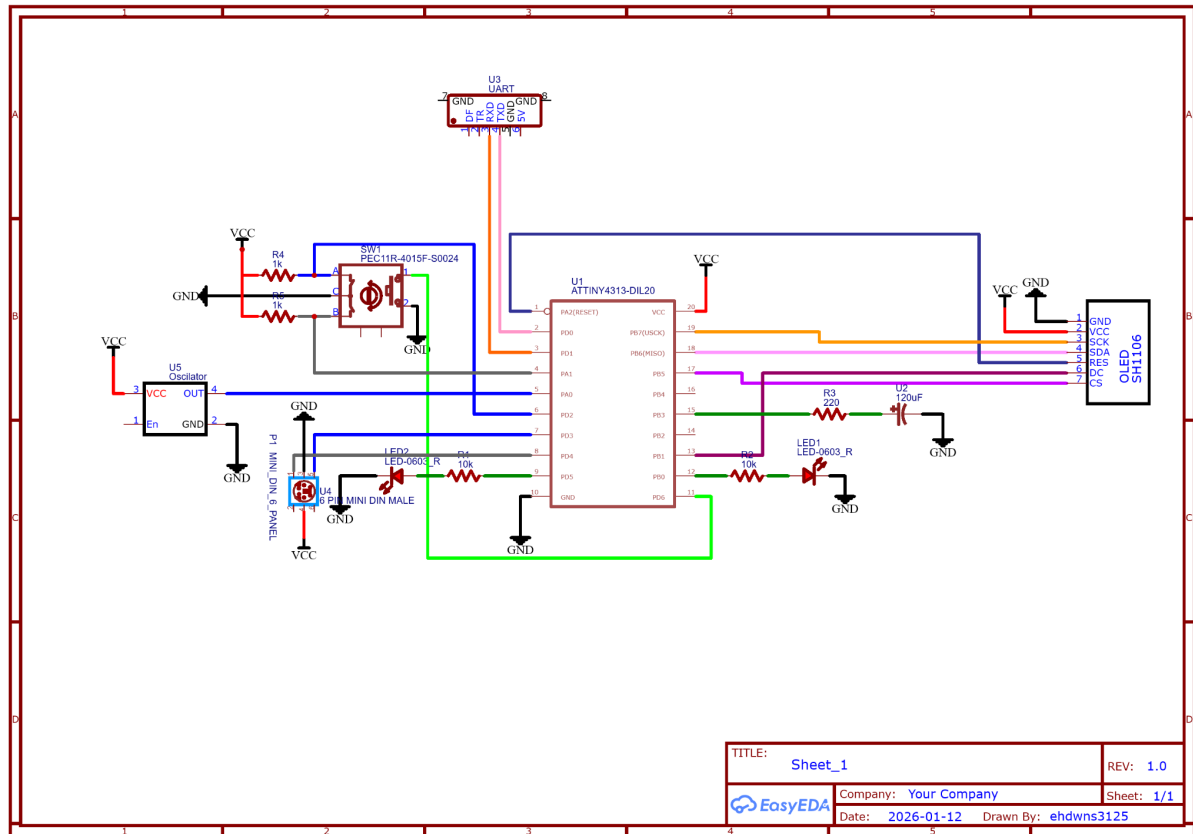
- 상태 관리: 사용자 입력에 따라 변경되는 입력 값 및 시스템 시간(sys_tick) 관리.
- 디스플레이 업데이트: counter 값의 변화에 따라 OLED 화면의 게이지(Gauge) UI를 실시간으로 갱신 (50ms 주기).
- 부저 피드백: 키보드 입력에 맞춰 부저를 통해 특정 음계(Scale)를 재생.
- PWM 제어: 엔코더 스위치 또는 키보드 입력을 통해 PWM을 설정하여 LED 조명의 밝기 조절.
- 관리자 콘솔: UART 통신을 통해 시스템 내부 상태(Tick, Counter, Duty)를 모니터링.

1.2. 입력 장치

- Rotary Encoder
 - 기능: 연속적인 값 조절 및 선택
 - 조작 방식:
 - 회전: counter 값을 증가/감소시켜 OLED 게이지 수치 조절
 - 스위치 클릭: 현재 counter 값을 PWM 출력(밝기)으로 확정(Set)
 - 3초간 입력이 없으면 조작 의도가 없는 것으로 판단, 원래 값으로 돌아감.
- PS/2 Keyboard
 - 기능: 직관적인 수치 입력 및 연주
 - 조작 방식: 숫자 키('0'~'9'): 즉시 해당 비율(0% ~ 100%)로 counter 및 PWM 값 설정, 각 숫자에 매핑된 음계(Tone) 재생을 통해 입력 피드백 제공

1.3. 출력 장치

- OLED Display (SH1106)
 - 기능: 그래픽 사용자 인터페이스(GUI) 표시
 - 표시 내용:
 - 게이지 테두리 및 게이지 바 (Gauge Bar): counter 값에 비례하여 채워짐
 - 전구 아이콘 (Bulb Icon): 화면 장식 요소
- Buzzer
 - 기능: 청각적 피드백
 - 출력 내용:
 - 키보드 숫자키 입력 시 해당 음계(도, 레, 미...) 재생
 - 일정 시간 후 자동으로 소리 꺼짐 제어
- PWM Output (LED)
 - 기능: 아날로그 출력 시뮬레이션
 - 출력 내용:
 - 사용자가 '설정'한 값에 따라 LED 밝기 변화 (0 ~ 100%)
- Status LED
 - 기능: 시스템 동작 확인 (Heartbeat)
 - 출력 내용:
 - 5초 간격으로 점멸하여 메인 루프가 정상 동작 중임을 알림
- UART Console
 - 기능: 디버깅 및 상태 확인
 - 출력 내용:
 - ls 명령어 입력 시: 현재 Ticks, counter reset까지 남은 ticks, Counter 값, PWM Duty 값 출력



2.2. pin map

pin	장치 연결
VCC	VCC
PB7 (USCK/SCL/SCK/PCINT7)	OLED SCK
PB6 (MISO/DO/PCINT6)	OLED SDA
PB5 (MOSI/DI/SDA/PCINT5)	OLED CS
PB4 (OC1B/PCINT4)	
PB3 (OC1A/PCINT3)	Buzzer
PB2 (OC0A/PCINT2)	
PB1 (AIN1/PCINT1)	OLED DC
PB0 (AIN0/PCINT0)	Heartbeat LED
PD6 (ICPI/PCINT17)	Encoder SW
(PCINT10/RESET/dW) PA2	RST
(PCINT11/RXD) PD0	UART TX
(PCINT12/TXD) PD1	UART RX
(PCINT9/XTAL2) PA1	Encoder DT
(PCINT8/CLKI/XTAL1) PA0	External CLK
(PCINT13/CKOUT/XCK/INT0) PD2	Encoder CLK
(PCINT14/INT1) PD3	PS2 CLK
(PCINT15/T0) PD4	PS2 DT
(PCINT16/OC0B/T1) PD5	PWM LED
GND	GND

3. 자원 활용 전략

3.1. 핀 절약 가능성 분석

- 입력 장치

1차 과제에서 분석 했듯, 명확히 **timing**으로 분리되는 출력 장치들과 달리, 입력장치 **switch**는 사람이 직접적으로 조작하므로 **system** 입장에서는 **random**한 것이나 다름이 없다. 따라서 **input**과 **output**을 공유하면 시스템 안정성을 해칠 수 있다. 그렇기 때문에 입력 장치 위주인 2차 프로젝트에서는 시스템의 핀 절약 전략이 제한된다.

- HW 예약 기능

USI SPI, Timer 및 PWM, UART 같은 기능은 GPIO가 아닌 내부적인 HW block으로 구성되어있으므로 정해진 **pin**에서 사용하는 것이 중요하다. 그렇기 때문에 일반적인 GPIO처럼 다른 핀을 사용하거나, 다른 방법으로 공유하는 것이 제한된다.

3.2. HW 자원의 효과적인 운용

- Timer0 공유

해당 프로젝트에서 PWM을 두 가지(LED pwm, buzzer tone)나 사용중이므로, 내부의 timer를 모두 사용했다. 여기서 timer interrupt를 통해서 내부 tick을 만들어 관리하기 위해서는 timer interrupt가 필요했다. 이를 위해서 timer의 TCNT가 max까지 올라가는 Fast pwm mode를 통하여 timer overflow interrupt와 PWM을 동시 구현하였다. 이를 통해서 한 개의 timer를 통해서 PWM을 쓰면서도 tick기반의 non-blocking delay 시스템을 구현하였다.

4. 소프트웨어 특징

4.1. non-blocking 구조

`delay_ms` 와 같은 단순 딜레이가 전체 시스템을 멈추는 것을 지양하고, 1차에서 사용했던 단순 `count`에 비해서 높은 수준의 시간 정확도를 구현하기 위해서 `timer interrupt`와 그에 따라 증가하는 `system tick`을 사용하였다. 이와 함께 `F_CPU`와 분주 모드 등에 맞춰 `1tick`과 `1.024ms`에 대한 변환을 쉽게 하기 위해서 `MS_TO_TICKS(m)` `TICKS_TO_MS(t)` 매크로를 구현하여 컴파일 타임에 원하는 `ms`에 최대한 가까운 `tick`으로 변환한다.

모든 작업(LED 깜빡임, 스위치 확인, OLED 업데이트)은 `sys_tick`과 마지막 실행 시간을 비교하여 "때가 되었는지"를 확인하고 실행한다. 따라서, 일정한 시간값을 기준으로 실행되는 동작(`heart beat`, `beep`, 화면 업데이트 등)의 여러 작업을 수행하면서도 '동시에 처리하는 것처럼' 보이게 만든다.

4.2. 인터럽트 기반 Input

CPU가 주요 입력 값을 항상 확인하는 `polling` 대신 인터럽트를 기반으로 입력이 동작한다. 이를 통해서 `main loop`는 시스템의 시간값 기반으로하거나 입력값이 존재하는 경우에만 동작한다. 그리고 예측 불가능한 시점에 발생하는 사람의 입력에는 즉각 반응하여 정확히 입력을 감지한다. 이렇게 했을 때 OLED 화면을 그리는 동작 같은 느린 동작 중간에 입력이 들어와도 즉각 `interrupt`가 반응하여 전체 시스템이 효율적이고 빠르게 동작하도록 만든다.

5. 한계 및 개선 가능성

5.1. SW 구조

UART 통신 효율화: 현재 송신 함수가 길이가 긴 문자열을 보내지만, **blocking** 방식이라 긴 메시지 전송시 시스템 지연 우려가 있다. 출력 버퍼를 도입하고 출력 부분 역시 시간 기반으로 동작하도록 구현하면 한 글자 보내고 다른 작업으로 넘어가는 등 **non-blocking**하게 동작할 수 있는 수준이 높아질 것이다.

5.2. 추가 핀 절약

CS 핀 절약 : 현재 **SPI** 라인을 **OLED** 하나만 사용하므로, 여러가지 슬레이브 기기가 공유되는 경우를 위해서 만들어진 **CS** 핀을 **GND**에 고정하여 항상 **active**로 만들면 핀을 절약할 수 있다. 실제로도 고정 결과 큰 이상이 있지는 않았지만, 이렇게 하면 항상 입력을 받고 있어 예기치 않는 입력으로 클럭이 틀어지거나 의도하지 않는 동작을 하는 등 노이즈에 취약할 수 있으므로, 안정적인 동작을 위해서는 추가적인 조치가 필요할 것이다.